

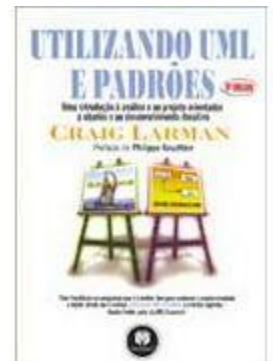
Utilizando UML e Padrões

Uma introdução à análise e ao projeto orientados a objetos
e ao desenvolvimento iterativo

Craig Larman

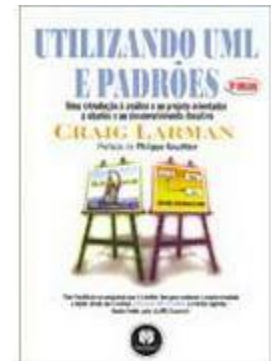
“Com frequência me perguntam qual é o melhor livro para conhecer o projeto orientado a objeto. Desde que o conheci, *Utilizando UML e Padrões* é a minha sugestão.”

Martin Fowler, autor de UML Essencial



Utilizando UML e Padrões

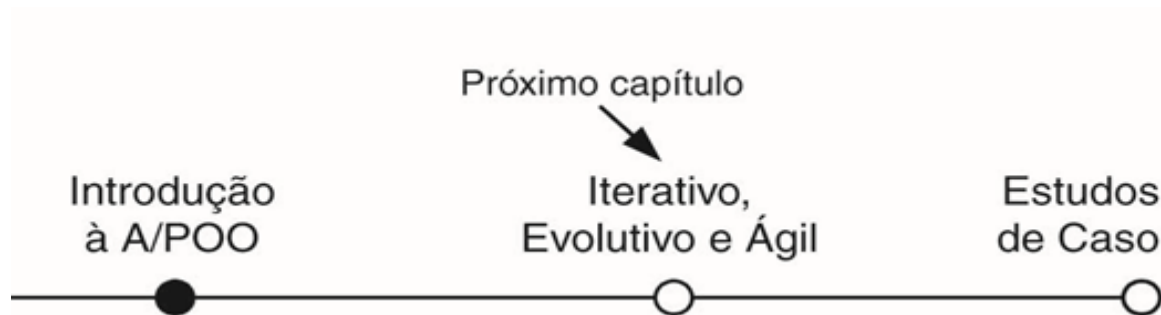
Capítulo 1 – Análise e Projeto Orientados a Objetos



Objetivos

- Descrever as metas e o escopo deste curso
- Definir **análise e projeto orientados a objetos** (A/POO)
- Ilustrar um breve exemplo de **A/POO**.
- Dar uma visão geral da **UML** e da **modelagem visual ágil**

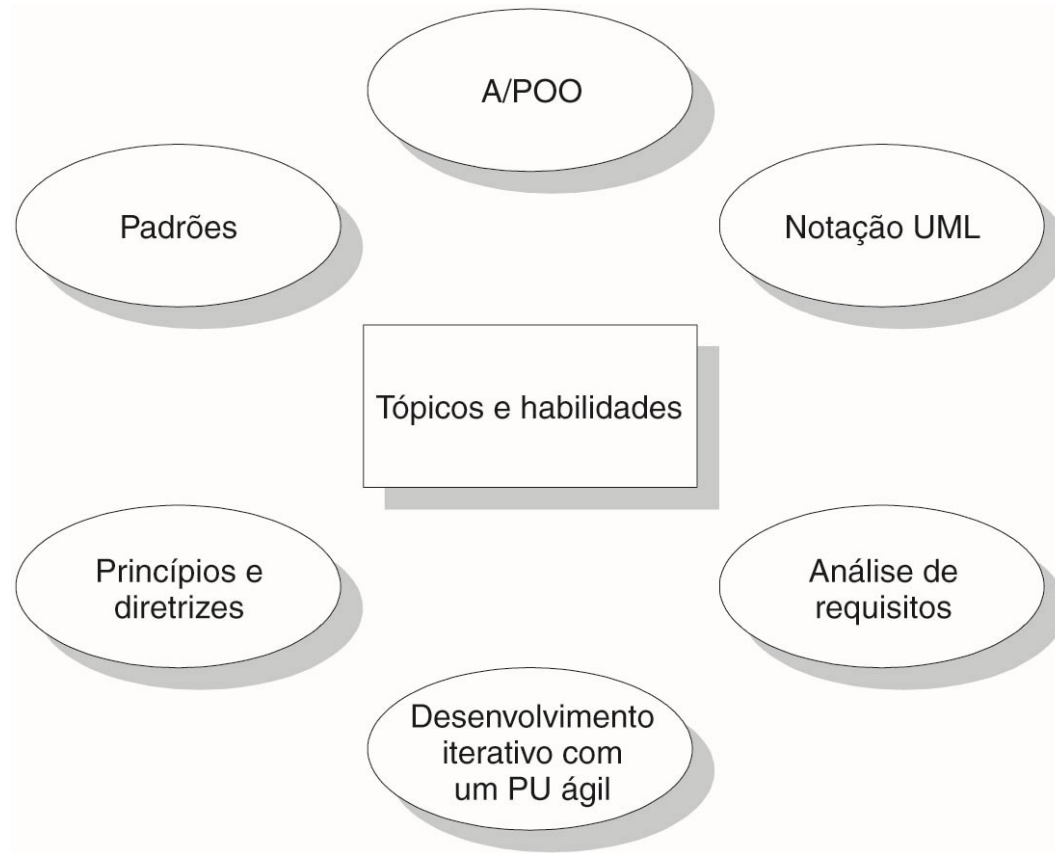
O que vem a seguir?



O que você aprenderá?

- **UML** vs. Pensar em objetos
- **POO**: princípios e padrões
- Estudos de caso
- **Casos de uso**
- **Desenvolvimento iterativo, modelagem ágil e uma abordagem ágil ao Processo Unificado (PU)**

Tópicos e habilidades abordadas



Qual o objetivo mais importante do aprendizado?

“Uma habilidade crucial no desenvolvimento OO é atribuir, adequadamente, responsabilidades aos objetos de software”.

O que são análise e projeto?

- **Análise**

- Enfatiza uma investigação do problema e dos requisitos em vez de uma solução

- **Projeto**

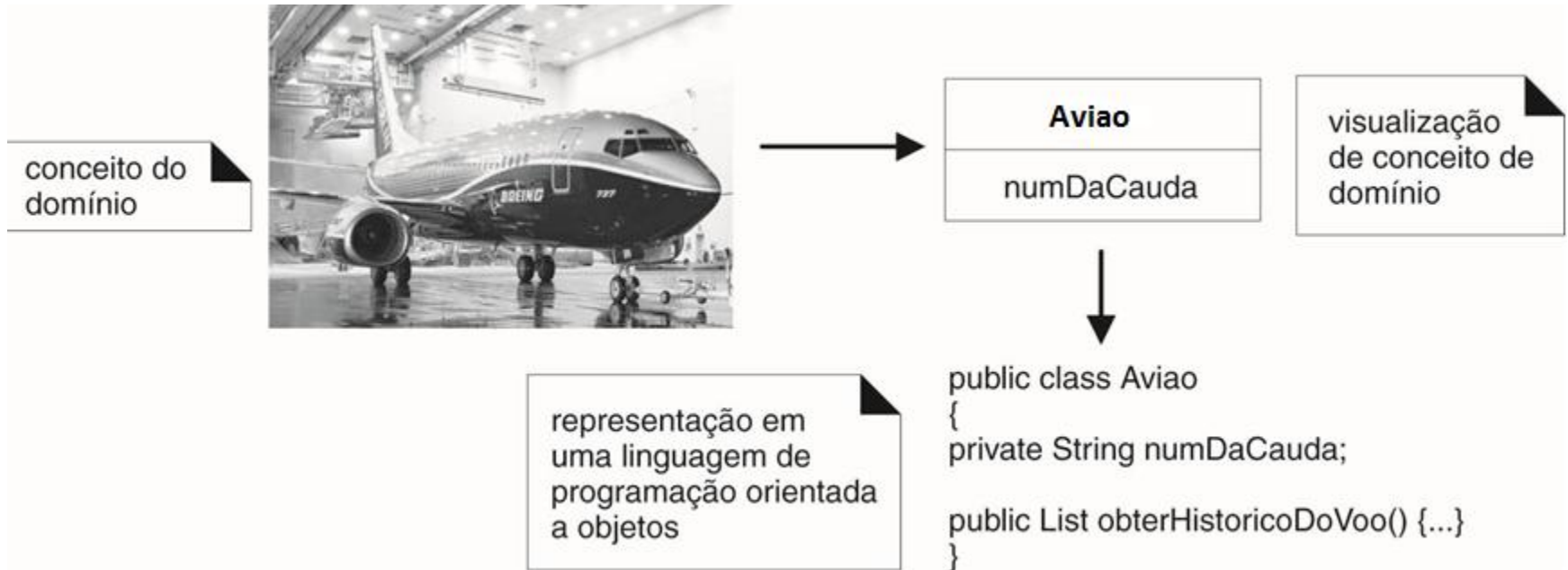
- Enfatiza uma solução conceitual em software ou hardware que satisfaça os requisitos

“Faça a coisa certa (análise) e faça certo a coisa (projeto)”

O que são análise e projeto orientados a objetos?

- **Análise orientada a objetos**
 - Ênfase em encontrar e descrever os conceitos no domínio do problema
- **Projeto orientado a objetos**
 - Ênfase na definição dos objetos de software e como eles colaboram para a satisfação dos requisitos.

O que são análise e projeto orientados a objetos?



Um processo básico através de um exemplo simples

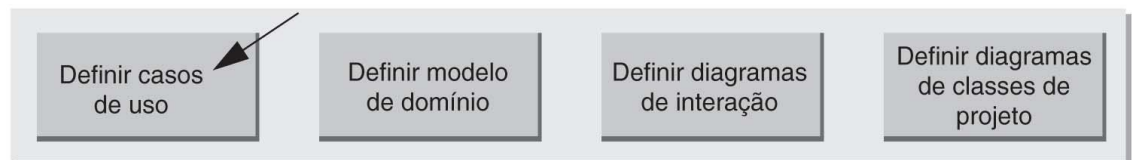
- **Exemplo:**
 - Um **jogo de dados** no qual um jogador lança dois dados.
 - Se o total for sete, ele vence; caso contrário, perde



Um processo básico através de um exemplo simples

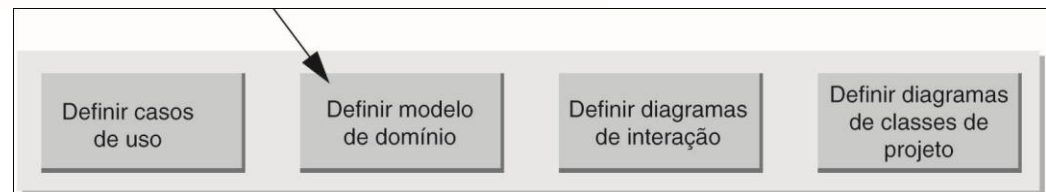
- **Casos de uso** são narrativas ou cenários sobre como as pessoas usam a aplicação

Jogar um jogo de dados: um jogador pede que os dados sejam lançados. O sistema apresenta o resultado: se a soma do valor das faces dos dados totalizar sete, ele vence; caso contrário, perde.



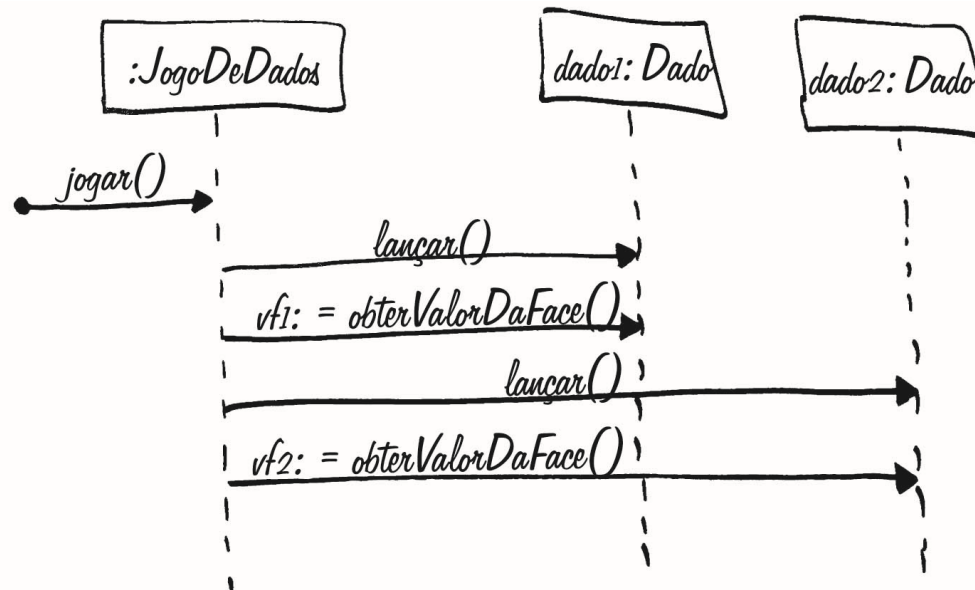
Um processo básico através de um exemplo simples

- Um **modelo de domínio** mostra os conceitos ou objetos do domínio que são de interesse.
 - Não é uma descrição dos objetos de software e sim **conceitos do mundo real**



Um processo básico através de um exemplo simples

- Um **projeto orientado a objetos** define objetos de software, suas **responsabilidades** e **colaborações**
 - Normalmente ilustrado com um **Diagrama de Seqüência**



Onde está o jogador?

Definir casos
de uso

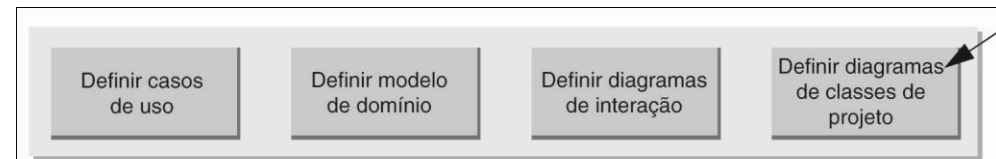
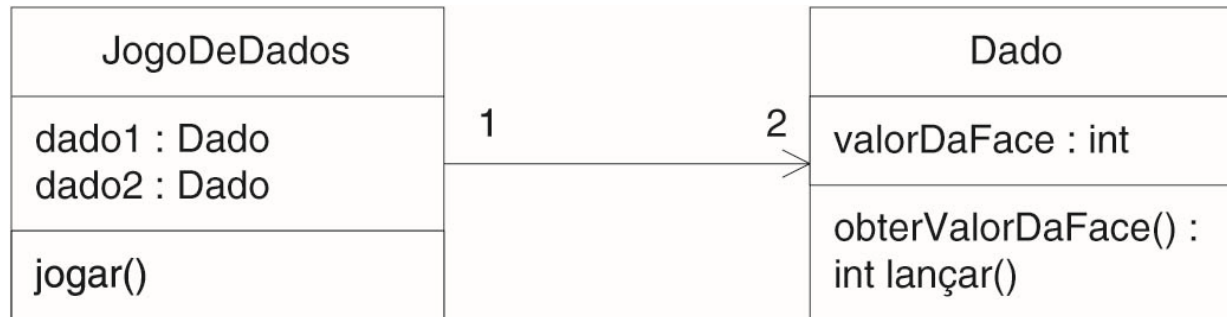
Definir modelo
de domínio

Definir diagramas
de interação

Definir diagramas
de classes de
projeto

Um processo básico através de um exemplo simples

- Um **diagrama de classes de projeto** ilustra **atributos e métodos das classes**.
 - Diferentemente do **modelo de domínio**, focado em classes do mundo real, este diagrama mostra as **classes de software**



O que é UML?

*“A Linguagem de Modelagem Unificada (UML) é uma **linguagem visual** para especificar, construir e documentar os artefatos dos sistemas”. [OMG03]*

Três modos de aplicar UML

- **UML como rascunho**
 - Muito rápida, sem necessidade de ferramentas
 - Preferida por “**modeladores ágeis**”.
 - Frequentemente suficiente.
- **UML como modelo de software**
 - Modelo detalhado confeccionado por ferramenta.
 - Usado para gerar código.
 - Engenharia reversa a partir do código.

Três modos de aplicar UML

- **UML como linguagem de programação**
 - Preparar um conjunto completo de modelos do sistema
 - Compilar modelos diretamente em código executável
 - Elimina a necessidade de codificação.
 - Para saber mais:
 - Vide esforço da **OMG** com o **MDA – Model Driven Architecture**

Três perspectivas para aplicar a UML

1. Perspectiva conceitual

- Diagramas descrevendo coisas do mundo real ou domínio de interesse.

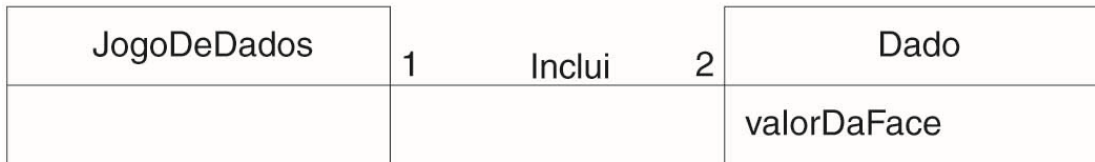
2. Perspectiva de especificação (software)

- Diagramas descrevendo abstrações de software com especificações e interfaces, mas sem comprometimento com uma linguagem de programação em particular.
- Pouco usada na prática.

3. Perspectiva de implementação (software)

- Diagramas descrevem implementações de software em uma tecnologia em particular, tal como **Java**.

Três perspectivas para aplicar a UML



Perspectiva conceitual
(modelo do domínio)

Notação do esboço do
Diagrama de Classe UML
usada para visualizar os
conceitos do mundo real



Perspectiva de
especificação ou
implementação (diagrama
de classe de projeto)

Notação do Diagrama de
Classe UML bruto usado
para visualizar os elementos
de software

UML e a idéia da “bala de prata”

- Idéia publicada no livro “*The Mythical Man Month*” de **Frederick Brooks**
 - Explora a idéia do erro fundamental em acreditar que existe uma ferramenta ou técnica especial que irá resolver todos os problemas no desenvolvimento de software.
- A **UML** é apenas uma notação de diagramação
 - Apesar de ajudar, saber sua notação não é tão importante quanto **saber projetar e pensar em termos de objetos.**



Histórico

- **Décadas de 60 e 70**
 - Surgimento de linguagens OO como **Simula** e **SmallTalk**
 - **Alan Kay**, criador do SmallTalk, cunhou os termos **programação orientada a objetos** e **computação pessoal**
- **Década de 80**
 - **Grady Booch** escreve o artigo ***“Object Oriented Design”***, em 1982
 - **Bertrand Meyer** publica o livro ***“Object-Oriented Software Construction”*** em 1988.
 - Em 1988, **Steve Mellor** e **Schlaer** publicam o livro ***“Object Oriented System Analysis”***, cunhando o termo **análise orientada a objetos**.

Histórico

- **Década de 90**

- Em 1990 e 1991, **Peter Coad** publicou os famosos livros ***“Object Oriented Analysis e Object Oriented Design”***
- Também em 1990, **Rebecca Wirfs-Brock** e outros descrevem a abordagem de projeto guiado por responsabilidades no livro ***“Designing Object Oriented Software”***.
- Em 1992, **Ivar Jacobson** publicou o livro ***“Object Oriented Software Engineering”*** que ficou famoso pelo uso de **casos de uso** para requisitos.

Histórico

- **Década de 90 (Cont.)**

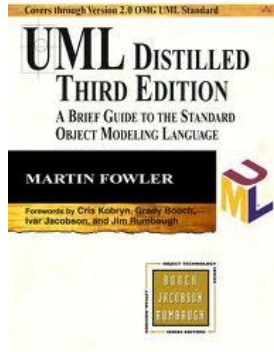
- Os três amigos (**Booch, Rumbaugh e Jacobson**) criam uma notação comum de diagramação conhecida como a **UML**.
- Em 1997, a **OMG** publica a **UML** como padrão aberto. Surge a **UML 1.0**

- **Atualmente**

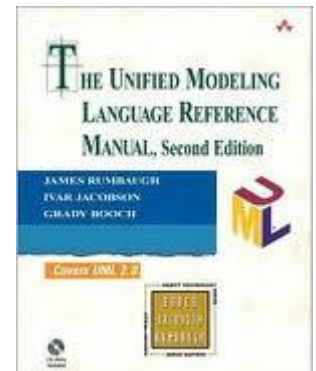
- A **UML** continua sendo refinada até os dias de hoje, através de novas versões disponíveis em www.omg.org ou www.uml.org

Leituras recomendadas

- **UML Distilled**
 - Martin Fowler



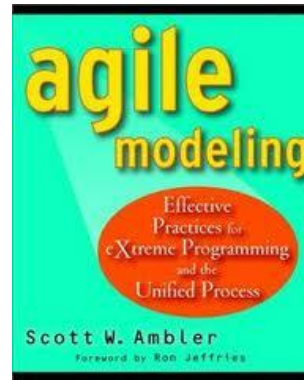
- **The Unified Modeling Language Reference Manual**
 - James Rumbaugh, Ivar Jacobson, Grady Booch



Leituras recomendadas

- **Agile Modeling**

- Scott Ambler



- **Design Patterns**

- Erich Gamma et. al.

